

El código de Golay extendido (24, 12)

Trabajo Práctico 2 · Curso de FEC 2026 · Fundación Fulgor

Villar, Pedro

Parte A — Propiedades del código

Datos

Matriz B (simétrica, $B^2 = I$), idéntica para todos los grupos:

fila	hex	binario	fila	hex	binario
b_0	98F	1001 1000 1111	b_6	53D	0101 0011 1101
b_1	4E7	0100 1110 0111	b_7	2BE	0010 1011 1110
b_2	357	0011 0101 0111	b_8	87B	1000 0111 1011
b_3	BE2	1011 1110 0010	b_9	E74	1110 0111 0100
b_4	DD1	1101 1101 0001	b_{10}	F1A	1111 0001 1010
b_5	7CC	0111 1100 1100	b_{11}	EA9	1110 1010 1001

Convención de bits: $cw[23:12]$ = mensaje, $cw[11:0]$ = paridad; el bit $m[11 - i]$ es m_i y el bit $[11 - j]$ de b_i es $B[i][j]$. Con esa convención, multiplicar un vector x de 12 bits por B es sumar (XOR) las filas b_i tales que $x_i = 1$:

$$x \cdot B = \bigoplus_{i: x_i=1} b_i.$$

Ejercicio 1 — Matrices y parámetros

G y H

$G = [I_{12} \mid B]$ y $H = [B \mid I_{12}]$, ambas 12×24 :

G			H		
g_0	100000000000	100110001111	h_0	100110001111	100000000000
g_1	010000000000	010011100111	h_1	010011100111	010000000000
g_2	001000000000	001101010111	h_2	001101010111	001000000000
g_3	000100000000	101111100010	h_3	101111100010	000100000000
g_4	000010000000	110111010001	h_4	110111010001	000010000000
g_5	000001000000	011111001100	h_5	011111001100	000001000000
g_6	000000100000	010100111101	h_6	010100111101	000000100000
g_7	000000010000	001010111110	h_7	001010111110	000000010000
g_8	000000001000	100001111011	h_8	100001111011	000000001000
g_9	000000000100	111001110100	h_9	111001110100	000000000100
g_{10}	000000000010	111100011010	h_{10}	111100011010	000000000010
g_{11}	000000000001	111010101001	h_{11}	111010101001	000000000001

B es simétrica

Se lee cada columna j de B de arriba a abajo y se compara con la fila b_j . El bit i de la columna j es $B[i][j]$, o sea el bit $[11 - j]$ de b_i :

j	columna j	b_j	j	columna j	b_j
0	1001 1000 1111	1001 1000 1111	6	0101 0011 1101	0101 0011 1101
1	0100 1110 0111	0100 1110 0111	7	0010 1011 1110	0010 1011 1110
2	0011 0101 0111	0011 0101 0111	8	1000 0111 1011	1000 0111 1011
3	1011 1110 0010	1011 1110 0010	9	1110 0111 0100	1110 0111 0100
4	1101 1101 0001	1101 1101 0001	10	1111 0001 1010	1111 0001 1010
5	0111 1100 1100	0111 1100 1100	11	1110 1010 1001	1110 1010 1001

Las doce coinciden, entonces $B^T = B$.

$$G H^T = 0$$

Trasponiendo H queda $H^T = \begin{bmatrix} B^T \\ I_{12} \end{bmatrix}$, y haciendo el producto por bloques:

$$G H^T = \begin{bmatrix} I_{12} & B \end{bmatrix} \begin{bmatrix} B^T \\ I_{12} \end{bmatrix} = I_{12} B^T + B I_{12} = B^T + B = B + B = 0,$$

Como B es simétrica (lo vimos en el punto anterior), B^T es lo mismo que B . Y en $GF(2)$ sumar algo con sí mismo da cero, porque la suma es un XOR, así que $B + B = 0$.

Chequeo numérico sobre una fila: $g_0 = (u_0 \mid b_0)$, entonces $g_0 H^T = u_0 B^T + b_0 = b_0 + b_0 = 0$, y lo mismo para las otras once.

$$B^2 = I$$

Como B es simétrica, $B^2 = B B^T$, así que la entrada (i, j) es el producto interno de dos *filas*:

$$(B^2)_{ij} = \sum_{k=0}^{11} B[i][k] B[k][j] = b_i \cdot b_j = w(b_i \wedge b_j) \text{ mód } 2,$$

o sea la paridad de la cantidad de posiciones donde b_i y b_j tienen ambos un 1. Como B^2 también es simétrica, alcanza con calcular la mitad de la tabla (78 entradas).

Tres entradas desarrolladas:

$$\begin{aligned} b_0 &= 1001\ 1000\ 1111 \\ b_0 &= 1001\ 1000\ 1111 \\ b_0 \wedge b_0 &= 1001\ 1000\ 1111 \implies w = 7 \equiv 1 \pmod{2} \end{aligned}$$

$$\begin{aligned} b_0 &= 1001\ 1000\ 1111 \\ b_1 &= 0100\ 1110\ 0111 \\ b_0 \wedge b_1 &= 0000\ 1000\ 0111 \implies w = 4 \equiv 0 \pmod{2} \end{aligned}$$

$$\begin{aligned} b_3 &= 1011\ 1110\ 0010 \\ b_7 &= 0010\ 1011\ 1110 \\ b_3 \wedge b_7 &= 0010\ 1010\ 0010 \implies w = 4 \equiv 0 \pmod{2} \end{aligned}$$

La tabla completa de $w(b_i \wedge b_j)$ (en gris, las entradas repetidas por simetría):

$w(b_i \wedge b_j)$	0	1	2	3	4	5	6	7	8	9	10	11
0	7	4	4	4	4	4	4	4	4	2	4	4
1	4	7	4	4	4	4	4	4	4	4	2	4
2	4	4	7	4	4	4	4	4	4	4	4	2
3	4	4	4	7	4	4	2	4	4	4	4	4
4	4	4	4	4	7	4	4	2	4	4	4	4
5	4	4	4	4	4	7	4	4	2	4	4	4
6	4	4	4	2	4	4	7	4	4	4	4	4
7	4	4	4	4	2	4	4	7	4	4	4	4
8	4	4	4	4	4	2	4	4	7	4	4	4
9	2	4	4	4	4	4	4	4	4	7	4	4
10	4	2	4	4	4	4	4	4	4	4	7	4
11	4	4	2	4	4	4	4	4	4	4	4	7

La diagonal vale $w(b_i) = 7$ en los doce casos, que es impar, y todas las entradas de fuera de la diagonal son pares. Tomando paridad queda $(B^2)_{ii} = 1$ y $(B^2)_{ij} = 0$ para $i \neq j$, es decir $B^2 = I_{12}$.

Dimensión, tasa y overhead

$$\begin{aligned} n &= 24, & k &= 12, & n - k &= 12 \quad (\text{bits de paridad}) \\ R &= \frac{k}{n} = \frac{12}{24} = \frac{1}{2} = 0,5 \\ \text{overhead} &= \frac{n - k}{k} = \frac{12}{12} = 1 = 100 \% \quad \left(\text{o bien } \frac{n - k}{n} = 50 \% \text{ del bloque} \right) \end{aligned}$$

El código es $(24, 12)$: entran 12 bits de mensaje y salen 24 de palabra código. Hay $2^{12} = 4096$ palabras código.

Codificación a mano de $m = \text{A5C}$

$$m = \text{A5C} = 1010\ 0101\ 1100 \implies m_i = 1 \text{ para } i \in \{0, 2, 5, 7, 8, 9\}$$

La paridad es $p = m \cdot B$, el XOR de esas seis filas:

$$\begin{array}{r} b_0 \ 1001\ 1000\ 1111 \\ \oplus \ b_2 \ 0011\ 0101\ 0111 \\ \hline 1010\ 1101\ 1000 \\ \oplus \ b_5 \ 0111\ 1100\ 1100 \\ \hline 1101\ 0001\ 0100 \\ \oplus \ b_7 \ 0010\ 1011\ 1110 \\ \hline 1111\ 1010\ 1010 \\ \oplus \ b_8 \ 1000\ 0111\ 1011 \\ \hline 0111\ 1101\ 0001 \\ \oplus \ b_9 \ 1110\ 0111\ 0100 \\ \hline p \ 1001\ 1010\ 0101 = 9\text{A5} \end{array}$$

$$v = (m \mid p) = 1010\ 0101\ 1100\ 1001\ 1010\ 0101 = \text{A5C9A5}$$

Verificación $v H^T = 0$

Partiendo $v = (m \mid p)$ y usando otra vez $H^T = \begin{bmatrix} B^T \\ I \end{bmatrix}$:

$$v H^T = m B^T + p I = m B + p = p + p = 0.$$

Numéricamente, $m \cdot B = 9\text{A5}$ (la cuenta de arriba) y $p = 9\text{A5}$, entonces

$$1001\ 1010\ 0101 \oplus 1001\ 1010\ 0101 = 0000\ 0000\ 0000 = 000. \quad \checkmark$$

Ejercicio 2 — Distancia mínima y capacidad de corrección

Distribución de pesos del código (dato del enunciado):

peso w	0	8	12	16	24
cantidad de palabras	1	759	2576	759	1

El código es lineal, así que la distancia mínima es el peso mínimo no nulo:

$$d_{\min} = \min\{w(v) : v \in C, v \neq 0\} = 8.$$

(Chequeo: $1 + 759 + 2576 + 759 + 1 = 4096 = 2^{12}$, están todas las palabras.)

Corrección. Para corregir t errores las esferas de radio t alrededor de cada palabra código no se tienen que tocar, lo que pide $d_{\min} \geq 2t + 1$:

$$2t + 1 \leq 8 \implies t \leq \frac{7}{2} = 3,5 \implies t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor = \left\lfloor \frac{7}{2} \right\rfloor = 3.$$

Detección. Usando el código solo para detectar, cualquier patrón de error de peso $\leq d_{\min} - 1 = 7$ no puede llevar una palabra código a otra, así que se detectan hasta **7** errores.

Los dos a la vez. Corrigiendo t y detectando $l \geq t$ hace falta $d_{\min} \geq t + l + 1$:

$$l \leq d_{\min} - t - 1 = 8 - 3 - 1 = 4.$$

O sea: corrige 3 y detecta 4. Esto se ve en el Ejercicio 3 con r_3 , que tiene un patrón de error de peso 4: el decodificador no lo corrige, pero lo marca como no corregible en lugar de entregar una palabra equivocada.

Ejercicio 3 — Decodificación manual

Algoritmo (Parte C). Con $s = r_{[23:12]} B \oplus r_{[11:0]}$ y $q = s \cdot B$, se prueban los casos en orden y gana el primero que se cumple:

caso	condición	patrón de error
1	$w(s) \leq 3$	$e = (0 \mid s)$
2	$\exists i : w(s \oplus b_i) \leq 2$	$e = (u_i \mid s \oplus b_i)$
3	$w(q) \leq 3$, con $q = s \cdot B$	$e = (q \mid 0)$
4	$\exists i : w(q \oplus b_i) \leq 2$	$e = (q \oplus b_i \mid u_i)$
5	ninguna de las anteriores	o_uncorrectable

u_i es el vector de 12 bits con un uno en la posición i . Los empates en i se resuelven por prioridad de índice (gana el i más chico).

$$r_1 = \text{A5D9A6}$$

$$r_{[23:12]} = \text{A5D} = 1010\ 0101\ 1101, \text{ con unos en } i \in \{0, 2, 5, 7, 8, 9, 11\}; r_{[11:0]} = \text{9A6}.$$

$ \begin{array}{rcl} & b_0 & 1001\ 1000\ 1111 \\ \oplus & b_2 & 0011\ 0101\ 0111 \\ \hline & & 1010\ 1101\ 1000 \\ \oplus & b_5 & 0111\ 1100\ 1100 \\ \hline & & 1101\ 0001\ 0100 \\ \oplus & b_7 & 0010\ 1011\ 1110 \\ \hline & & 1111\ 1010\ 1010 \\ \oplus & b_8 & 1000\ 0111\ 1011 \\ \hline & & 0111\ 1101\ 0001 \\ \oplus & b_9 & 1110\ 0111\ 0100 \\ \hline & & 1001\ 1010\ 0101 \\ \oplus & b_{11} & 1110\ 1010\ 1001 \\ \hline & r_{[23:12]} B & 0111\ 0000\ 1100 = 70C \end{array} $	$ \begin{array}{rcl} & r_{[23:12]} B & 0111\ 0000\ 1100 \\ \oplus & r_{[11:0]} & 1001\ 1010\ 0110 \\ \hline & s & 1110\ 1010\ 1010 = \text{EAA} \end{array} $
--	--

$w(s) = 7 > 3$, el caso 1 no aplica. Se buscan los doce candidatos $s \oplus b_i$:

i	b_i	$s \oplus b_i$	w
0	1001 1000 1111	0111 0010 0101	6
1	0100 1110 0111	1010 0100 1101	6
2	0011 0101 0111	1101 1111 1101	10
3	1011 1110 0010	0101 0100 1000	4
4	1101 1101 0001	0011 0111 1011	8
5	0111 1100 1100	1001 0110 0110	6
6	0101 0011 1101	1011 1001 0111	8
7	0010 1011 1110	1100 0001 0100	4
8	1000 0111 1011	0110 1101 0001	6
9	1110 0111 0100	0000 1101 1110	6
10	1111 0001 1010	0001 1011 0000	4
11	1110 1010 1001	0000 0000 0011	2 ←

Salta $i = 11$ con peso 2, entonces es el **caso 2**:

$$e = (u_{11} \mid s \oplus b_{11}) = (0000\ 0000\ 0001 \mid 0000\ 0000\ 0011) = 001003, \quad w(e) = 3.$$

$$\begin{array}{r} r \quad 1010\ 0101\ 1101\ 1001\ 1010\ 0110 \\ \oplus e \quad 0000\ 0000\ 0001\ 0000\ 0000\ 0011 \\ \hline v \quad 1010\ 0101\ 1100\ 1001\ 1010\ 0101 = \text{A5C9A5} \end{array}$$

Mensaje decodificado: $\hat{m} = \text{A5C}$, que es el mismo del Ejercicio 1.

$$r_2 = \text{A5F9A4}$$

$$r_{[23:12]} = \text{A5F} = 1010\ 0101\ 1111, \text{ con unos en } i \in \{0, 2, 5, 7, 8, 9, 10, 11\}; r_{[11:0]} = \text{9A4}.$$

$$\begin{array}{r} b_0 \quad 1001\ 1000\ 1111 \\ \oplus b_2 \quad 0011\ 0101\ 0111 \\ \hline 1010\ 1101\ 1000 \\ \oplus b_5 \quad 0111\ 1100\ 1100 \\ \hline 1101\ 0001\ 0100 \\ \oplus b_7 \quad 0010\ 1011\ 1110 \\ \hline 1111\ 1010\ 1010 \\ \oplus b_8 \quad 1000\ 0111\ 1011 \\ \hline 0111\ 1101\ 0001 \\ \oplus b_9 \quad 1110\ 0111\ 0100 \\ \hline 1001\ 1010\ 0101 \\ \oplus b_{10} \quad 1111\ 0001\ 1010 \\ \hline 0110\ 1011\ 1111 \\ \oplus b_{11} \quad 1110\ 1010\ 1001 \\ \hline r_{[23:12]} B \quad 1000\ 0001\ 0110 = 816 \end{array} \quad \begin{array}{r} r_{[23:12]} B \quad 1000\ 0001\ 0110 \\ \oplus r_{[11:0]} \quad 1001\ 1010\ 0100 \\ \hline s \quad 0001\ 1011\ 0010 = 1B2 \end{array}$$

$w(s) = 5 > 3$: no es el caso 1. Candidatos $s \oplus b_i$:

i	b_i	$s \oplus b_i$	w
0	1001 1000 1111	1000 0011 1101	6
1	0100 1110 0111	0101 0101 0101	6
2	0011 0101 0111	0010 1110 0101	6
3	1011 1110 0010	1010 0101 0000	4
4	1101 1101 0001	1100 0110 0011	6
5	0111 1100 1100	0110 0111 1110	8
6	0101 0011 1101	0100 1000 1111	6
7	0010 1011 1110	0011 0000 1100	4
8	1000 0111 1011	1001 1100 1001	6
9	1110 0111 0100	1111 1100 0110	8
10	1111 0001 1010	1110 1010 1000	6
11	1110 1010 1001	1111 0001 1011	8

Ninguno baja de peso 2, así que tampoco es el caso 2. Se pasa a $q = s \cdot B$:

$$\begin{array}{rcl}
& b_3 & 1011\ 1110\ 0010 \\
\oplus & b_4 & 1101\ 1101\ 0001 \\
\hline
& & 0110\ 0011\ 0011 \\
\oplus & b_6 & 0101\ 0011\ 1101 \\
\hline
& & 0011\ 0000\ 1110 \\
\oplus & b_7 & 0010\ 1011\ 1110 \\
\hline
& & 0001\ 1011\ 0000 \\
\oplus & b_{10} & 1111\ 0001\ 1010 \\
\hline
q & & 1110\ 1010\ 1010 = \mathbf{EAA}
\end{array}$$

$w(q) = 7 > 3$: no es el caso 3. Candidatos $q \oplus b_i$:

i	b_i	$q \oplus b_i$	w
0	1001 1000 1111	0111 0010 0101	6
1	0100 1110 0111	1010 0100 1101	6
2	0011 0101 0111	1101 1111 1101	10
3	1011 1110 0010	0101 0100 1000	4
4	1101 1101 0001	0011 0111 1011	8
5	0111 1100 1100	1001 0110 0110	6
6	0101 0011 1101	1011 1001 0111	8
7	0010 1011 1110	1100 0001 0100	4
8	1000 0111 1011	0110 1101 0001	6
9	1110 0111 0100	0000 1101 1110	6
10	1111 0001 1010	0001 1011 0000	4
11	1110 1010 1001	0000 0000 0011	2 ←

Aparece $i = 11$ con peso 2, entonces es el **caso 4**:

$$e = (q \oplus b_{11} \mid u_{11}) = (0000\ 0000\ 0011 \mid 0000\ 0000\ 0001) = 003001, \quad w(e) = 3.$$

$$\begin{array}{rcl}
& r & 1010\ 0101\ 1111\ 1001\ 1010\ 0100 \\
\oplus & e & 0000\ 0000\ 0011\ 0000\ 0000\ 0001 \\
\hline
& v & 1010\ 0101\ 1100\ 1001\ 1010\ 0101 = \mathbf{A5C9A5}
\end{array}$$

Mensaje decodificado: $\hat{m} = \mathbf{A5C}$, otra vez el del Ejercicio 1.

$$r_3 = \text{A5C9AA}$$

$r_{[23:12]} = \text{A5C}$, con unos en $i \in \{0, 2, 5, 7, 8, 9\}$ (es el mismo mensaje del Ejercicio 1), y $r_{[11:0]} = \text{9AA}$.

$$\begin{array}{rcl}
 & b_0 & 1001\ 1000\ 1111 \\
 \oplus & b_2 & 0011\ 0101\ 0111 \\
 \hline
 & & 1010\ 1101\ 1000 \\
 \oplus & b_5 & 0111\ 1100\ 1100 \\
 \hline
 & & 1101\ 0001\ 0100 \\
 \oplus & b_7 & 0010\ 1011\ 1110 \\
 \hline
 & & 1111\ 1010\ 1010 \\
 \oplus & b_8 & 1000\ 0111\ 1011 \\
 \hline
 & & 0111\ 1101\ 0001 \\
 \oplus & b_9 & 1110\ 0111\ 0100 \\
 \hline
 & r_{[23:12]} B & 1001\ 1010\ 0101 = \text{9A5}
 \end{array}
 \qquad
 \begin{array}{rcl}
 & r_{[23:12]} B & 1001\ 1010\ 0101 \\
 \oplus & r_{[11:0]} & 1001\ 1010\ 1010 \\
 \hline
 & s & 0000\ 0000\ 1111 = \text{00F}
 \end{array}$$

$w(s) = 4 > 3$: no es el caso 1. Candidatos $s \oplus b_i$:

i	b_i	$s \oplus b_i$	w
0	1001 1000 1111	1001 1000 0000	3
1	0100 1110 0111	0100 1110 1000	5
2	0011 0101 0111	0011 0101 1000	5
3	1011 1110 0010	1011 1110 1101	9
4	1101 1101 0001	1101 1101 1110	9
5	0111 1100 1100	0111 1100 0011	7
6	0101 0011 1101	0101 0011 0010	5
7	0010 1011 1110	0010 1011 0001	5
8	1000 0111 1011	1000 0111 0100	5
9	1110 0111 0100	1110 0111 1011	9
10	1111 0001 1010	1111 0001 0101	7
11	1110 1010 1001	1110 1010 0110	7

Ninguno con peso ≤ 2 : tampoco es el caso 2. Se calcula $q = s \cdot B$:

$$\begin{array}{rcl}
 & b_8 & 1000\ 0111\ 1011 \\
 \oplus & b_9 & 1110\ 0111\ 0100 \\
 \hline
 & & 0110\ 0000\ 1111 \\
 \oplus & b_{10} & 1111\ 0001\ 1010 \\
 \hline
 & & 1001\ 0001\ 0101 \\
 \oplus & b_{11} & 1110\ 1010\ 1001 \\
 \hline
 & q & 0111\ 1011\ 1100 = \text{7BC}
 \end{array}$$

$w(q) = 8 > 3$: no es el caso 3. Candidatos $q \oplus b_i$:

i	b_i	$q \oplus b_i$	w
0	1001 1000 1111	1110 0011 0011	7
1	0100 1110 0111	0011 0101 1011	7
2	0011 0101 0111	0100 1110 1011	7
3	1011 1110 0010	1100 0101 1110	7
4	1101 1101 0001	1010 0110 1101	7
5	0111 1100 1100	0000 0111 0000	3
6	0101 0011 1101	0010 1000 0001	3
7	0010 1011 1110	0101 0000 0010	3
8	1000 0111 1011	1111 1100 0111	9
9	1110 0111 0100	1001 1100 1000	5
10	1111 0001 1010	1000 1010 0110	5
11	1110 1010 1001	1001 0001 0101	5

Ninguno con peso ≤ 2 : tampoco es el caso 4. Se llega al **caso 5**, `o_uncorrectable` en alto, y $\hat{m}$ y e no son válidas.

Comparando con la palabra del Ejercicio 1, $v = \text{A5C9A5}$:

$$r_3 \oplus v = \text{A5C9AA} \oplus \text{A5C9A5} = 00000F = 0000\ 0000\ 0000\ 0000\ 0000\ 1111, \quad w = 4.$$

El error real tiene peso 4, uno más que $t = 3$. Es justo el caso del Ejercicio 2: el código lo detecta pero no lo puede corregir.

Parte B — Modelo de referencia en software

El modelo de referencia está en el paquete `golay/` y usa la librería `fec_algebra` del TP1. Los resultados de este informe salen de correr `scripts/caracterizacion.py`.

Ejercicio 4 — $GF(2^m)$ con $m = 1$

Se instancia el campo del TP1 con $m = 1$ y $P(x) = x + 1$: `GF(m=1, primitive_poly=0b1)`. Queda un campo de dos elementos con estas tablas:

$+$	0	1	$\cdot$	0	1	a	a^{-1}
0	0	1	0	0	0	0	$—$
1	1	0	1	0	1	1	1

La suma es XOR y el producto es AND, o sea que es $GF(2)$. No hubo que modificar la librería. El producto con $m = 1$ hace una sola iteración y da $a \wedge b$, y el inverso se calcula como $a^{2^m-2} = a^0 = 1$, que es correcto porque el único elemento invertible es el 1. El 0 tira `GFZeroDivisionError`.

Lo que faltaba era el álgebra de matrices, que está en `golay/linalg.py`: `vec_mat` (producto vector-matriz), `mat_mat` (producto matriz-matriz) y `weight` (peso de Hamming), más algunas auxiliares (`transpose`, `identity`, `hstack`, `vec_add`). Sirven para cualquier $GF(2^m)$, no solo para $GF(2)$.

Ejercicio 5 — Construcción y verificación

B se carga con las doce filas del enunciado y el resto se arma con `linalg`: $G = \text{hstack}(I, B)$, $H = \text{hstack}(B, I)$ y $H^T = \text{transpose}(H)$. Las propiedades del Ejercicio 1 se verifican con los productos matriciales:

```
>>> from golay import *
>>> from golay import linalg
>>> linalg.equal(B, linalg.transpose(B)) # B^T = B
True
>>> linalg.equal(linalg.mat_mat(B, B), linalg.identity(12, GF2)) # B^2 = I
True
>>> linalg.is_zero(linalg.mat_mat(G, HT)) # G H^T = 0
True
```

Da lo mismo que en la Parte A.

Codificando los 4096 mensajes con $v = m \cdot G$ y contando pesos:

peso w	0	8	12	16	24	total
modelo	1	759	2576	759	1	4096
enunciado	1	759	2576	759	1	4096

Coincide con la tabla del Ejercicio 2, y el peso mínimo no nulo da $d_{\min} = 8$.

Ejercicio 6 — Codificador y decodificador

El codificador es $v = m \cdot G$ con `vec_mat`:

```
>>> hex(encode(0xA5C))
'0xa5c9a5'
>>> hex(syndrome(0xA5C9A5))
'0x0'
```

Da la misma palabra que a mano en el Ejercicio 1, y su síndrome es nulo.

El decodificador tiene una función por cada submódulo del RTL de la Parte C, así después cada testbench compara contra su función:

RTL	modelo	etapa
golay_syndrome	syndrome	1
golay_mult_b	mult_b	2
popcount12	popcount12	2
golay_row_search	row_search	2
golay_err_gen	err_gen	3
golay_correct	correct	3

`decode()` llama a esas funciones en el orden del pipeline. Como `golay_correct` solo recibe `i_rx` e `i_err`, `o_corrected` es simplemente $e \neq 0$. Cuando `row_search` no encuentra fila, `o_idx` y `o_res` no importan.

`decode()` devuelve el mensaje, el patrón de error y las dos banderas, `corrected` y `uncorrectable`, y además el síndrome y el caso, que sirven para comparar con la Parte A. Decodificando las tres palabras del Ejercicio 3 (síndrome, caso, error, mensaje y las dos banderas):

```
>>> for r in (0xA5D9A6, 0xA5F9A4, 0xA5C9AA):
...     d = decode(r)
...     print(hex(d.syndrome), d.case, hex(d.err), hex(d.msg), d.corrected, d.uncorrectable)
...
0xea 2 0x1003 0xa5c True False
0x1b2 4 0x3001 0xa5c True False
0xf 5 0x0 0x0 False True
```

Coincide con lo que dio a mano en la Parte A: r_1 se resuelve en el caso 2, r_2 en el caso 4 y r_3 es no corregible.

Tabla de síndromes

La tabla se arma recorriendo todos los errores de peso ≤ 4 , calculando su síndrome con H y guardando, para cada síndrome, los patrones de peso mínimo:

peso mínimo	patrones de ese peso	síndromes	
0	1	1	corregible
1	1	24	corregible
2	1	276	corregible
3	1	2024	corregible
4	6	1771	no corregible
			4096

Los 2325 síndromes que tienen un solo patrón de peso mínimo se corrigen. A cada uno de los 1771 restantes le corresponden seis patrones de peso 4, y no hay forma de saber cuál fue, así que se declaran no corregibles.

El algoritmo de cuatro casos no usa esta tabla: decide solo con los pesos y las filas b_i . Por eso la tabla se usa para verificarlo: decodificar buscando el síndrome en la tabla es otro método, independiente del de cuatro casos, y para todos los errores de peso ≤ 4 los dos dan el mismo resultado.

Ejercicio 7 — Caracterización

Errores de peso ≤ 4

Se decodifican todos los patrones de error de peso 0 a 4 sobre la palabra nula (alcanza con esa porque el síndrome solo depende de e):

peso de e	0	1	2	3	4
patrones	1	24	276	2024	10626
corregidos bien	1	24	276	2024	0
decodificados a otra palabra	0	0	0	0	0
detectados como no corregibles	0	0	0	0	10626

Coincide con la tabla del enunciado. En ningún caso se decodifica a una palabra equivocada.

Estímulo para la Parte C

No se generan archivos de vectores: los testbenches de cocotb importan `golay/stimulus.py` y generan el estímulo ahí mismo, como en la Unidad 3. Hay un generador por módulo, y cada uno devuelve diccionarios cuyas claves son los nombres de los puertos del módulo, con las entradas y las salidas esperadas. Las claves que empiezan con `_` no son puertos sino datos extra, como `_case`, el caso en el que cae cada palabra. Un testbench los usa así:

```
for vec in mult_b_vectors():
    dut.i_vec.value = vec["i_vec"]
    await Timer(1, "ns")
    assert int(dut.o_vec.value) == vec["o_vec"]
```

Los módulos de 12 bits se prueban con las 4096 entradas. El decodificador completo tiene 2^{24} entradas posibles, así que se usa una muestra: las 4096 palabras código, todos los errores de peso 1 a 3 sobre cinco palabras, y parte de los de peso 4.

módulo	casos		caso	vectores	condición
golay_mult_b	4096	exhaustivo	1	5586	$w(s) \leq 3$
popcount12	4096	exhaustivo	2	4740	$\exists i : w(s \oplus b_i) \leq 2$
golay_row_search	4096	exhaustivo	3	1430	$w(q) \leq 3$
golay_encoder	4096	exhaustivo	4	3960	$\exists i : w(q \oplus b_i) \leq 2$
golay_syndrome	6420	palabras código + errores	5	110	no corregible
golay_err_gen	15826	las del decodificador			
golay_correct	15826	las del decodificador			
golay_decoder	15826	muestra, cubre los 5 casos			

Preguntas

¿Por qué el síndrome depende solo del error?

Con $r = v + e$:

$$s = r H^T = v H^T + e H^T = e H^T,$$

porque $v = m G$ y $G H^T = 0$. Por eso el síndrome no depende del mensaje.

¿Por qué $d_{\min} = 8$ detecta 4 errores pero no los corrige?

Detecta: si e pesa 4, $v + e$ no puede ser otra palabra código, porque la diferencia entre dos palabras código pesa al menos 8. Entonces el síndrome no es cero y el error se nota.

No corrige: para corregir t errores hace falta $d_{\min} \geq 2t + 1$, y con $d_{\min} = 8$ eso da $t = 3$. En la tabla de síndromes se ve por qué: un error de peso 4 cae en un síndrome que comparten seis patrones de peso 4, todos igual de probables, y no hay forma de saber cuál fue. Por eso el decodificador llega al caso 5 y levanta `o_uncorrectable`, que es lo que pasó con r_3 en la Parte A.

Parte C — Implementación RTL

El RTL está en `rtl/`, un archivo por módulo, con las interfaces del enunciado. Los testbenches están en `tb/` y se corren con `cocotb` e `Icarus` desde `sim/` (`make todos`).

Ejercicio 8 — Codificador

`golay_mult_b` calcula $x \cdot B$ como el XOR de las filas b_i donde $x_i = 1$, igual que en la cuenta a mano de la Parte A. Cada fila se enmascara con el bit correspondiente y se hace el XOR de las doce:

$$o_vec = (\{12\{i_vec[11]\}\} \& 12'h98F) \wedge \dots \wedge (\{12\{i_vec[0]\}\} \& 12'hEA9)$$

`popcount12` suma los doce bits en un `for` dentro de un `always_comb`.

`golay_encoder` es una instancia de `golay_mult_b` que calcula la paridad y un registro que guarda `{i_msg, paridad}`. Latencia de un ciclo.

Ejercicio 9 — Decodificador

Los cuatro submódulos son combinacionales:

- `golay_syndrome`: una instancia de `golay_mult_b` sobre $r[23:12]$ y un XOR con $r[11:0]$.
- `golay_row_search`: arma los doce candidatos $v \oplus b_i$ en paralelo con un `generate`, cada uno con su `popcount12`. Para no escribir la matriz de nuevo, cada fila b_i se obtiene pasando u_i por un `golay_mult_b`; como la entrada es fija, al sintetizar eso queda como constantes y no ocupa lógica. Después un `for` recorre los candidatos del 0 al 11 y se queda con el primero de peso ≤ 2 , que es el de menor índice.
- `golay_err_gen`: un `if/else if` con los cuatro casos en orden. Si no entra en ninguno (caso 5), levanta `o_uncorrectable` y deja el error en cero.
- `golay_correct`: $r \oplus e$, se queda con los 12 bits de arriba como mensaje, y `o_corrected` es el OR de los bits de e .

El decodificador une todo en tres etapas:

etapa	lógica	registros
E1	<code>golay_syndrome</code>	s, r
E2	$q = s \cdot B$, pesos y búsquedas de s y q	$s, q, w(s), w(q)$, las dos búsquedas, r
E3	<code>golay_err_gen, golay_correct</code>	salidas

En E2 también se registra s , que no figura en la tabla del enunciado, porque `golay_err_gen` lo necesita para el caso 1.

Ejercicio 10 — Verificación y análisis

Cada testbench toma el estímulo de `golay/stimulus.py` (Parte B) y compara la salida del RTL contra el modelo:

módulo	vectores	qué se verifica
<code>golay_mult_b</code>	4096	salida, y aplicado dos veces vuelve a la entrada ($B^2 = I$)
<code>popcount12</code>	4096	barrido exhaustivo
<code>golay_row_search</code>	4096	barrido exhaustivo
<code>golay_syndrome</code>	6420	síndrome nulo en las 4096 palabras código, más errores
<code>golay_err_gen</code>	15826	patrón de error y bandera
<code>golay_correct</code>	15826	palabra, mensaje y <code>o_corrected</code>
<code>golay_encoder</code>	4096	los 4096 mensajes, uno por ciclo
<code>golay_decoder</code>	15826	mensaje y error bit a bit, y las dos banderas

En `golay_row_search`, cuando no hay fila, `o_idx` y `o_res` no se comparan, y en el decodificador `o_msg` y `o_err` no se comparan cuando está `o_uncorrectable`.

Pasan los ocho. Esta es la línea de resultado que imprime cocotb para cada uno al correr `make todos`:

```

** TEST                                STATUS  SIM TIME (ns)  REAL TIME (s)  RATIO (ns/s) **
** test_golay_mult_b.test_mult_b      PASS      8192.00        3.12        2623.87 **
** test_popcount12.test_popcount12    PASS      4096.00        0.84        4884.76 **
** test_golay_row_search.test_row_search PASS      4096.00        7.02        583.22 **
** test_golay_syndrome.test_syndrome  PASS      6420.00        5.21        1232.60 **
** test_golay_err_gen.test_err_gen    PASS     15826.00       63.03        251.07 **
** test_golay_correct.test_correct    PASS     15826.00       43.49        363.92 **
** test_golay_encoder.test_encoder    PASS     40975.00       7.12        5752.52 **
** test_golay_decoder.test_decoder    PASS    158295.00      68.85        2299.06 **

```

El testbench del decodificador además imprime cuántos vectores cayeron en cada caso, y se ve que pasan por los cinco:

vectores: 15826, por caso: {1: 5586, 2: 4740, 3: 1430, 4: 3960, 5: 110}

Latencia. El codificador tiene latencia 1 y el decodificador 3, los dos con una palabra por ciclo. Los testbenches meten un vector por ciclo y comparan la salida 1 y 3 ciclos después; si la latencia fuera otra, fallarían.

Instancias. Dentro del decodificador:

submódulo	directas	dentro de otros	total
golay_syndrome	1	—	1
golay_row_search	2	—	2
golay_err_gen	1	—	1
golay_correct	1	—	1
golay_mult_b	1	1 en el síndrome, 12 en cada búsqueda	26
popcount12	2	12 en cada búsqueda	26

El codificador tiene una sola instancia de `golay_mult_b`. De las 26 del decodificador, 24 son las de las búsquedas, que tienen entrada constante y no generan lógica.

Parte D — Interleaver convolucional (bonus)

Ejercicio 11 — Interleaver convolucional

Los dos módulos están en `rtl-bonus/`. Cada rama es un shift register que solo avanza cuando el conmutador pasa por ella:

- `conv_interleaver`: la rama i tiene $i \cdot J$ bits. La rama 0 no tiene registro y pasa el bit directo.
- `conv_deinterleaver`: la rama i tiene $(\lambda - 1 - i) \cdot J$ bits, así que la que pasa directo es la última.

Las ramas se arman con un `generate` según los parámetros. El conmutador es un contador de 0 a $\lambda - 1$ que arranca en 0 después del reset y avanza una rama por ciclo. La salida es el bit de la rama seleccionada, registrado.

Como la rama i solo avanza una vez cada λ bits, un bit que entra por la rama i del interleaver sale $i \cdot J \cdot \lambda$ bits después. Pasando por los dos módulos, cada bit recorre $i \cdot J$ registros en uno y $(\lambda - 1 - i) \cdot J$ en el otro, así que todas las ramas quedan con el mismo retardo y el orden original se recupera:

$$D = (\lambda - 1) \cdot J \cdot \lambda + 2,$$

donde el $+2$ son los registros de salida de cada módulo. Con los valores por defecto ($\lambda = 24$, $J = 1$) da $D = 554$ bits.

La memoria total es $\sum_i i \cdot J = \frac{\lambda(\lambda-1)}{2} J$ bits en cada módulo: 276 bits con los valores por defecto.

Verificación

Se ponen el interleaver y el deinterleaver uno detrás del otro (`tb/conv_chain.sv`), se meten bits al azar, uno por ciclo, y se verifica que a la salida aparece la misma secuencia, atrasada D bits.

La salida del interleaver llega un ciclo tarde por su registro, así que el deinterleaver sale del reset un ciclo después para que los dos conmutadores estén en la misma rama cuando pasa cada bit.

Pasa con $\lambda = 24$, $J = 1$ y también con $\lambda = 5$, $J = 3$ (`make TOP=conv_chain LAMBDA=5 J=3`), donde $D = 62$:

```
LAMBDA=24 J=1, retardo 554 bits
** test_conv_chain.test_chain    PASS      16635.00      0.19      89801.30  **
LAMBDA=5 J=3, retardo 62 bits
** test_conv_chain.test_chain    PASS      1875.00      0.01     230533.73  **
```